VTC Academy
Think Ahead, Beyond Boundaries

Database Design & SQL Report

Library Management System

Class: K26ISTG01

Name: Tran Thi Huyen Trang

Student ID: 134010126013

Course: Database Design & SQL

1. Introduction

1.1 Project Overview

The Library Management System Database is an individual academic project developed to apply relational database design and SQL programming concepts. The project focuses on organizing library information, maintaining relationships between data entities, and retrieving meaningful information through SQL queries.

A library manages different types of information, including books, authors, publishers, categories, members, and borrowing transactions. Without a properly designed database, this information may become duplicated, inconsistent, and difficult to maintain.

This project uses MySQL as the database management system and MySQL Workbench as the development environment. The database is designed with eight related tables and includes sample data, indexes, a view, a stored procedure, a trigger, and SQL queries.

1.2 Project Objectives

The main objectives are:

- To analyze the information requirements of a library management system.
- To design an Entity-Relationship Diagram (ERD).
- To implement a relational database with primary keys and foreign keys.
- To apply database normalization principles up to Third Normal Form (3NF).
- To insert sample records and verify database operations.
- To write SQL queries using filtering, sorting, joins, aggregate functions, subqueries, window functions, pagination, and Common Table Expressions (CTEs).
- To implement database objects such as indexes, views, stored procedures, and triggers.
- To manage project files and documentation using GitHub.

1.3 Scope of the Project

The project covers the database layer of a library management system. It supports the storage and retrieval of book information, author and publisher details, member information, and borrowing records.

The project focuses on database design and SQL implementation rather than a complete application with a graphical user interface.

2. System Requirements Analysis

2.1 Functional Requirements

The database supports the following functional requirements:

1. Store information about authors, publishers, and book categories.
2. Store book information, including ISBN, title, publication year, and total copies.
3. Associate books with authors.
4. Store library member information.
5. Record borrowing transactions and their borrowing dates, due dates, and statuses.
6. Record the books included in each borrowing transaction.
7. Retrieve information from multiple related tables.
8. Summarize borrowing activities and book information using SQL queries.

2.2 Non-Functional Requirements

The database should maintain data integrity through appropriate primary keys, foreign keys, and column constraints. Its structure should reduce unnecessary data duplication and support clear, maintainable SQL scripts.

The project should also be reproducible: another user should be able to create the database, insert sample data, and execute the queries by following the instructions in the GitHub repository.

3. Database Design

3.1 Entity-Relationship Diagram

The database contains eight tables:

1. authors
2. book_authors
3. books
4. borrowing_details
5. borrowings
6. categories

7. members
8. publishers

The books table stores book information and references the corresponding publisher and category. The book_authors table represents the many-to-many relationship between books and authors. The borrowings table records borrowing transactions, while borrowing_details identifies the books associated with those transactions.

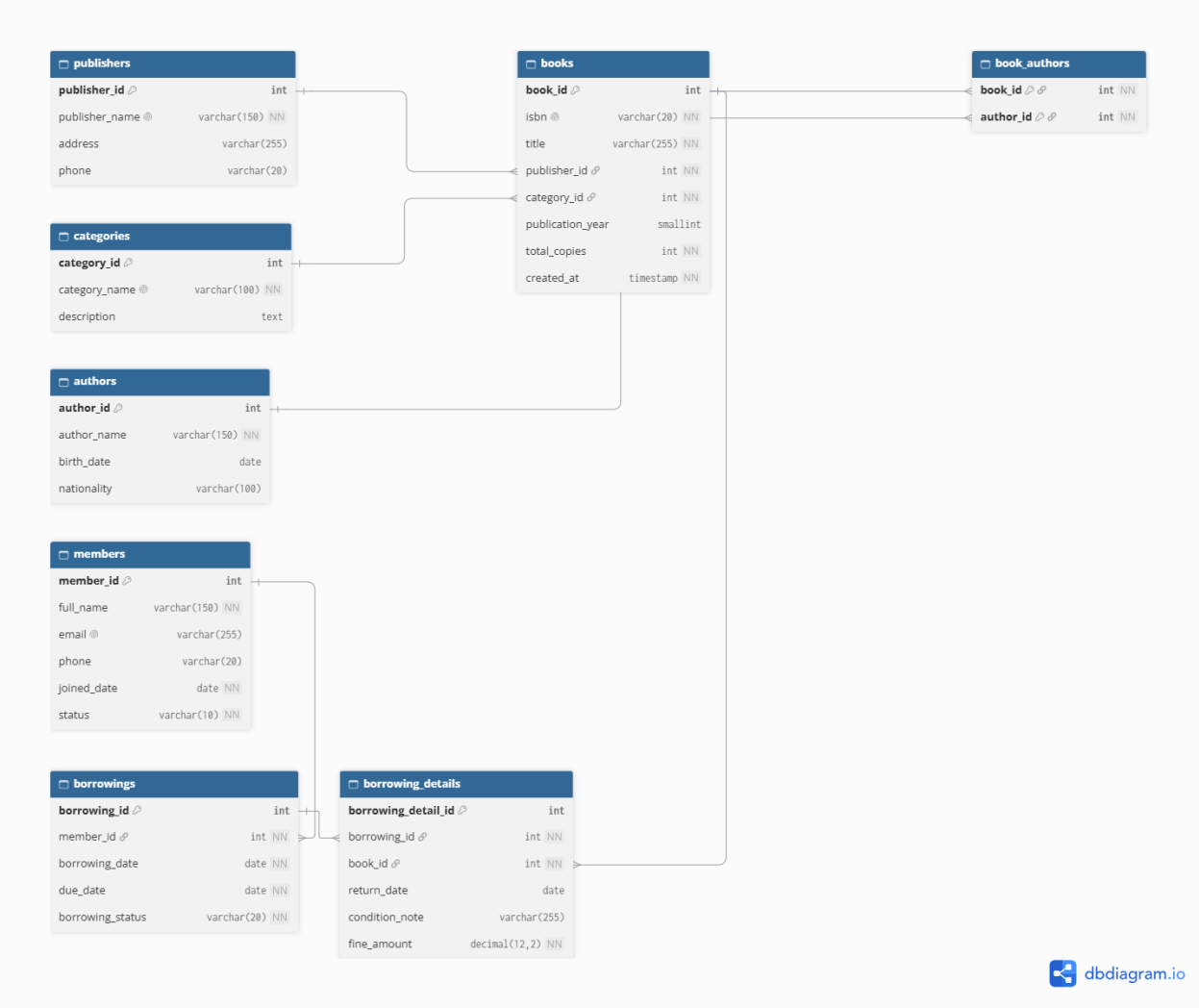


Figure 1. Entity-Relationship Diagram of the Library Management System Database.

3.2 Table Descriptions

Table	Purpose
authors	Stores author information.

publishers	Stores publisher information.
categories	Stores book category information.
books	Stores book details and references publishers and categories.
book_authors	Connects books with their authors.
members	Stores library member information and account status.
borrowings	Stores borrowing transactions, dates, and statuses.
borrowing_details	Stores individual book details associated with borrowing transactions, including return information and fines.

The database was populated with sample records for all eight tables. The sample data provides a basis for testing relationships, filtering conditions, joins, and aggregate calculations.

3.3 Primary Keys and Foreign Keys

Primary keys uniquely identify records within their respective tables. Foreign keys establish relationships between tables and help maintain referential integrity.

The main relationships include:

- books.publisher_id references publishers.publisher_id.
- books.category_id references categories.category_id.
- book_authors.book_id references books.book_id.
- book_authors.author_id references authors.author_id.
- borrowings.member_id references members.member_id.
- borrowing_details.borrowing_id references borrowings.borrowing_id.
- borrowing_details.book_id references books.book_id.

The book_authors table resolves the many-to-many relationship between books and authors. This design allows a book to have multiple authors and an author to be associated with multiple books.

3.4 Database Normalization

Normalization helps organize relational data, reduce redundancy, and prevent insertion, update, and deletion anomalies.

First Normal Form (1NF): Each table stores records in a structured format, with individual columns representing attributes rather than repeating groups. Relationships between books and authors are represented through the separate book_authors table.

Second Normal Form (2NF): Non-key attributes should depend on the entire primary key. The junction table book_authors represents the association between a book and an author without storing unrelated descriptive attributes in that relationship.

Third Normal Form (3NF): Non-key attributes should depend on the key, the whole key, and nothing but the key. Publisher and category information are stored in their own tables instead of being repeatedly stored as descriptive text in every book record. Member details and borrowing transaction details are also stored separately.

The schema follows these normalization principles in its overall structure. The final submission should verify the actual table definitions and functional dependencies to confirm 3NF compliance.

4. Database Implementation

4.1 Database Creation

The database was implemented in MySQL. SQL scripts were organized into separate files to distinguish database creation, sample data insertion, database objects, and query examples.

The implementation process involved creating the tables, defining relationships, inserting sample records, and testing the resulting database in MySQL Workbench.

4.2 Sample Data

Sample records were inserted into all eight tables. These records represent books, authors, publishers, categories, members, and borrowing transactions.

The sample data makes it possible to test both individual tables and relationships between multiple tables. It also provides input for the ten SQL queries developed in this project.

```
mysql> show tables;
+--------------------------+
| Tables_in_library_db     |
+--------------------------+
| authors                  |
| book_authors             |
| books                    |
| borrowing_details        |
| borrowings               |
| categories               |
| members                  |
| publishers               |
| v_book_borrowing_summary |
+--------------------------+
9 rows in set (0.00 sec)
```

Figure 2. SHOW TABLES output showing the eight tables.]

table_name	total_records
authors	10
book_authors	10
books	10
borrowing_details	10
borrowings	10
categories	10
members	10
publishers	10

Figure 3. Sample data or row counts for the eight tables.]

4.3 Indexes

Indexes were created to support searches and joins involving frequently accessed columns. The project includes indexes named idx_books_title, idx_borrowings_member_date, and idx_borrowing_details_book.

These indexes demonstrate how database indexes can support query execution. Their actual performance benefits depend on data volume, query conditions, and the execution plan selected by MySQL.

4.4 View

The view v_book_borrowing_summary combines book information with borrowing details. It summarizes the number of borrowing-detail records associated with each book and counts records that have not yet been returned.

The view simplifies access to summarized information without requiring the same aggregation query to be rewritten every time.

4.5 Stored Procedure

The stored procedure sp_member_borrowing_history accepts a member ID as an input parameter and retrieves the borrowing history associated with that member.

It joins member, borrowing, borrowing-detail, and book information to provide a consolidated result. This demonstrates the use of parameterized stored procedures for reusable database operations.

4.6 Trigger

The trigger trg_validate_return_date executes before a new record is inserted into borrowing_details. It checks whether a non-null return date is earlier than the borrowing date.

If the return date violates this rule, the trigger raises an SQL error and rejects the insertion. This demonstrates how triggers can enforce business rules at the database level.

5. SQL Queries and Results

Ten SQL queries were implemented and executed in MySQL Workbench. They demonstrate different SQL techniques for retrieving and analyzing library data.

5.1 Query 1: Sorting and Limiting Results

This query retrieves a limited number of books ordered by publication year in descending order. It demonstrates ORDER BY and LIMIT.

book_id	title	isbn	publication_year	total_copies
3	Sapiens	9780062316097	2011	6
9	Thinking, Fast and Slow	9780374533557	2011	3
4	Clean Code	9780132350884	2008	5
5	Harry Potter and the Deathly Hallows	9780545010221	2007	7
6	The Alchemist	9780062315007	1988	4
NULL	NULL	NULL	NULL	NULL

Figure 4. Query 1 SQL statement and result.

5.2 Query 2: Filtering Active Members

This query selects active members and sorts their information by name. It demonstrates conditional filtering with WHERE and sorting with ORDER BY.

member_id	full_name	email	phone
7	Bui Phuong Thao	phuongthao07@example.com	0901000007
6	Do Thanh Nam	thanhnam06@example.com	0901000006
5	Hoang Ngoc Linh	ngoclinh05@example.com	0901000005
3	Le Thu Ha	thuha03@example.com	0901000003
10	Ngo Hoang Long	hoanglong10@example.com	0901000010
1	Nguyen Minh Anh	minhanh01@example.com	0901000001
4	Pham Duc Huy	duchuy04@example.com	0901000004
2	Tran Quoc Bao	quocbao02@example.com	0901000002
8	Vu Tuan Kiet	tuankiet08@example.com	0901000008
NULL	NULL	NULL	NULL

Figure 5. Query 2 SQL statement and result

5.3 Query 3: INNER JOIN

This query combines books with their categories using INNER JOIN. It displays book titles together with category names.

book_id	title	category_name
5	Harry Potter and the Deathly Hallows	Children
10	The Little Prince	Children
3	Sapiens	History
1	1984	Literature
8	Murder on the Orient Express	Literature
2	Pride and Prejudice	Literature
6	The Alchemist	Literature
9	Thinking, Fast and Slow	Psychology
7	A Brief History of Time	Science
4	Clean Code	Technology

Figure 6. Query 3 SQL statement and result

5.4 Query 4: LEFT JOIN

This query combines books, the junction table book_authors, and authors using LEFT JOIN. It allows books to remain in the result even if they do not have an associated author record.

book_id	title	author_name
1	1984	George Orwell
7	A Brief History of Time	Stephen Hawking
4	Clean Code	Robert C. Martin
5	Harry Potter and the Deathly Hallows	J. K. Rowling
8	Murder on the Orient Express	Agatha Christie
2	Pride and Prejudice	Jane Austen
3	Sapiens	Yuval Noah Harari
6	The Alchemist	Paulo Coelho
10	The Little Prince	Antoine de Saint-Exupery
9	Thinking, Fast and Slow	Daniel Kahneman

Figure 7. Query 4 SQL statement and result

5.5 Query 5: Aggregate Functions and GROUP BY

This query groups borrowing transactions by status and uses COUNT() to calculate the number of transactions in each status group.

It demonstrates how aggregate functions can summarize transaction data.

borrowing_status	total_borrowings
returned	5
borrowing	3
overdue	2

Figure 8. Query 5 SQL statement and result

5.6 Query 6: Subquery

This query retrieves members whose IDs appear in the borrowings table. The IN operator and a subquery are used to identify members who have borrowing records.

member_id	full_name	email
7	Bui Phuong Thao	phuongthao07@example.com
6	Do Thanh Nam	thanhnam06@example.com
5	Hoang Ngoc Linh	ngoclinh05@example.com
3	Le Thu Ha	thuha03@example.com
1	Nguyen Minh Anh	minhanh01@example.com
4	Pham Duc Huy	duchuy04@example.com
2	Tran Quoc Bao	quocbao02@example.com
8	Vu Tuan Kiet	tuankiet08@example.com
NULL	NULL	NULL

Figure 9. Query 6 SQL statement and result

5.7 Query 7: CASE WHEN

This query classifies borrowing records using CASE WHEN. Depending on the borrowing status and due date, a record is assigned a descriptive label such as Completed, Overdue, Past Due, or In Progress.

borrowing_id	member_id	due_date	borrowing_status	status_description
1	1	2026-09-15	returned	Completed
2	2	2026-09-19	returned	Completed
3	3	2026-09-24	returned	Completed
4	4	2026-09-29	returned	Completed
5	5	2026-10-04	returned	Completed
6	6	2026-10-15	overdue	Overdue
7	7	2026-10-19	overdue	Overdue
8	8	2026-10-24	borrowing	In Progress
9	1	2026-10-29	borrowing	In Progress
10	2	2026-11-03	borrowing	In Progress

Figure 10. Query 7 SQL statement and result

5.8 Query 8: Window Function

This query uses RANK() to rank books by their number of borrowing-detail records, using the v_book_borrowing_summary view.

Window functions allow ranking calculations while retaining individual book records in the result.

book_id	title	total_borrow_records	borrowing_rank
1	1984	1	1
7	A Brief History of Time	1	1
4	Clean Code	1	1
5	Harry Potter and the Deathly Hallows	1	1
8	Murder on the Orient Express	1	1
2	Pride and Prejudice	1	1
3	Sapiens	1	1
6	The Alchemist	1	1
10	The Little Prince	1	1
9	Thinking, Fast and Slow	1	1

Figure 11. Query 8 SQL statement and result

5.9 Query 9: Pagination

This query retrieves the second page of book records, with five records per page. It uses LIMIT and OFFSET to demonstrate pagination.

book_id	title	isbn	publication_year
6	The Alchemist	9780062315007	1988
7	A Brief History of Time	9780553380163	1988
8	Murder on the Orient Express	9780062073488	1934
9	Thinking, Fast and Slow	9780374533557	2011
10	The Little Prince	9780156012195	1943
NULL	NULL	NULL	NULL

Figure 12. Query 9 SQL statement and result

5.10 Query 10: CTE and Nested Query

This query uses a Common Table Expression named member_borrowing_counts to calculate the number of borrowing transactions for each member. A nested query calculates the average borrowing count, and the outer query returns members whose counts exceed that average.

This demonstrates the use of CTEs and nested aggregate queries to make analytical SQL more readable.

member_id	full_name	total_borrowings
1	Nguyen Minh Anh	2
2	Tran Quoc Bao	2

Figure 13. Query 10 SQL statement and result

6. Testing and Results

Testing was performed using MySQL Workbench. The database creation scripts, sample data, database objects, and ten SQL queries were executed to verify their operation.

The main testing activities included:

- Verifying that all eight tables were created.
- Checking that sample records were available for querying.
- Executing joins between related tables.
- Testing filtering, sorting, grouping, and aggregate calculations.
- Executing a stored procedure to retrieve a member's borrowing history.
- Confirming that the view returned summarized borrowing information.
- Inspecting the trigger to verify that it was created.

Screenshots of the SQL query results provide evidence of the implemented operations.

The tests demonstrate the intended database functionality with the sample dataset. They do not establish performance characteristics under large-scale workloads.

7. Project Organization and GitHub

Demo Video Link: https://youtu.be/haepsbiPsIY?si=q3i3SRQJPBvCGqdV

The video demonstrates how the queries retrieve and analyze data from the Library Management System database. The results help verify that the database supports common library management and reporting tasks.

The project is maintained in a GitHub repository:

https://github.com/huyentrang277/library-database-design-sql

The repository is intended to contain the database schema, sample data scripts, ERD files, SQL queries, documentation, and query result screenshots.

Separating the SQL scripts by purpose makes the project easier to understand and reproduce. The README file provides an overview of the project and instructions for running the scripts.

8. Challenges and Lessons Learned

During the project, several practical issues were encountered.

One challenge was maintaining consistency between SQL scripts and the actual database schema. For example, using an incorrect column name can cause an SQL error. These issues highlighted the importance of checking table definitions before executing queries.

Another challenge involved duplicate index names. Attempting to create an index that already existed resulted in a duplicate-key-name error. This demonstrated why database scripts should be organized carefully and why existing database objects should be checked before rerunning creation statements.

The project also provided practical experience with joins, aggregate functions, subqueries, CTEs, and database objects such as views, stored procedures, and triggers.

Using GitHub helped organize the project files and preserve the implementation history. Overall, the project strengthened my understanding of relational database design, SQL development, debugging, and technical documentation.

9. Conclusion

The Library Management System Database project demonstrates the practical application of relational database design and SQL programming using MySQL.

The implementation includes eight related tables, sample data, indexes, a view, a stored procedure, a trigger, and ten SQL queries covering a range of database concepts.

The project provided experience in translating a real-world information management problem into a structured relational schema, implementing database relationships, testing SQL statements, and documenting the results.

Future improvements could include more comprehensive data validation, additional borrowing and return rules, more extensive automated testing, and performance evaluation using a larger dataset.

In conclusion, the project meets its primary learning objectives by demonstrating database design, SQL implementation, query development, and project documentation.